ISO 9001:2000
ISO 27001:2005
CMM L5
CMMI L5 V1.2

Cranes Software International Limited

Introduction to Python Programming

Python

- Printing Messages on Screen
- Reading input from console
- Python Identifiers and Keywords
- Data types in Python
- Operators

- Developed by Guido Van Roussum in 1991.
- Python is processed at runtime by the interpreter.
- There is no need to compile your program before executing it.
- Python has a design philosophy that emphasizes code readability.

Hello World Program

```
>>> print('Hello')
>>> print('Welcome to Python Programming!')
```

Print Statement

#Python 2.x Code

```
print "SIKANDER"
print 5
print 2 + 4
```

#Python 3.x Code

```
print("SIKANDER")
print(5)
print(2 + 4)
```

Output

```
SIKANDER
5
6
```

- The "print" directive prints contents to console.
- In Python 2, the "print" statement is not a function, and therefore it is invoked without parentheses.
- In Python 3, it is a function, and must be invoked with parentheses.
- (It includes a newline, unlike in C)
- A hash sign (#) that is not inside a string literal begins a comment.
- All characters after the # and up to the end of the physical line are part of the comment and the Python interpreter ignores them.

Python Identifiers

- Identifier is the name given to entities like class, functions, variables etc.
- **Rules for writing identifiers**
- Identifiers can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore (_).
- An identifier cannot start with a digit.
- Keywords cannot be used as identifiers.
- We cannot use special symbols like !, @, #, $, % etc. in our identifier.

Python Keywords

- Keywords are the reserved words in Python.
- We cannot use a keyword as variable name, function name or any other identifier.
- They are used to define the syntax and structure of the Python language.
- keywords are case sensitive.
- There are 33 keywords in Python 3.3.

Keywords in Python programming language				
False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

All the keywords except **True**, **False** and **None** are in lowercase

Python Variables

- A variable is a location in memory used to store some data (value).
- They are given unique names to differentiate between different memory locations. The rules for writing a variable name is same as the rules for writing identifiers in Python.
- We don't need to declare a variable before using it.
- In Python, we simply assign a value to a variable and it will exist.
- We don't even have to declare the type of the variable. This is handled internally according to the type of value we assign to the variable.

Variable assignment

- We use the assignment operator (=) to assign values to a variable.
- Any type of value can be assigned to any valid variable.

```
>>> regno = 1
>>> name = "Sikander"
>>> marks = 56.54
>>>
>>> print(regno, name, marks)
1 Sikander 56.54
```

Multiple assignments

- In Python, multiple assignments can be made in a single statement as follows:

```
>>> regno, name = 1, "Sikander"
>>> print(regno)
1
>>> print(name)
Sikander
```

If we want to assign the same value to multiple variables at once, we can do this

```
x = y = z = "same"
```

Data types in Python

- Every value in Python has a datatype.
- Since everything is an object in Python
- Data types are actually classes and variables are instance (object) of these classes.

- Numbers
  - int
  - float
- String – str
- Boolean - bool

Type of object – type()

- type() function can be used to know to which class a variable or a value belongs to.

```
print (type(regno))
print (type(name))
print (type(marks))
```

```
<class 'int'>
<class 'str'>
<class 'float'>
```

Reading Input

If you want to *prompt* the user for input, you can use `raw_input` in Python 2.X, and just `input` in Python 3.

Python 2.x

```
print("Enter the name :")
name = raw_input()
print(name)
```

Python 3.x

```
print("Enter the name :")
name = input()
print(name)
```

- Input function can print a prompt and read the input.

- name = input("Enter the name :")
- print(“Welcome “ + name)

```
name = input("Enter the name :")
print("Welcome " + name)
```

```
Enter the name :
```

Arithmetic Operators

Operator	Meaning	Example
+	Add two operands or unary plus	x + y +2
-	Subtract right operand from the left or unary minus	x - y -2
*	Multiply two operands	x * y
/	Divide left operand by the right one (always results into float)	x / y
%	Modulus - remainder of the division of left operand by the right	x % y (remainder of x/y)
//	Floor division - division that results into whole number adjusted to the left in the number line	x // y
**	Exponent - left operand raised to the power of right	x**y (x to the power y)

Arithmetic Operators

```
1    x = 15
2    y = 4
3
4    # Output: x + y = 19
5    print('x + y =',x+y)
6
7    # Output: x - y = 11
8    print('x - y =',x-y)
9
10   # Output: x * y = 60
11   print('x * y =',x*y)
12
13   # Output: x / y = 3.75
14   print('x / y =',x/y)
15
16   # Output: x // y = 3
17   print('x // y =',x//y)
18
19   # Output: x ** y = 50625
20   print('x ** y =',x**y)
```

What is the output

```
a = input("Enter value of a : ")
b = input("Enter value of b : ")
print (a + b)
```

```
Enter value of a : 4
Enter value of b : 5
```

- Data read from **input** is in the form of string.
- To convert to integer, we need to typecast.
- Syntax : datatype(object)

```
a = input("Enter value of a : ")
b = input("Enter value of b : ")
print (int(a) + int(b) )
```

```
Enter value of a : 4
Enter value of b : 5
9
```

first.py - C:/Users/PPS/Desktop/sastha/first.py (3.6.3)

File Edit Format Run Options Window Help

```
print("Enter the product Cost " )
productCost = input()
productCost = int(productCost)
gstPercent = 12

gstAmount = gstPercent / 100 * productCost
totalAmount = productCost + gstAmount

print(productCost)
print(gstAmount)
print(totalAmount)
```

Exercise

- Write a program to calculate the area of triangle given its radius.

- area = πr^2

```
#Program to calculate the area of circle

radius = input('Enter the radius of Circle: ')
area = 3.14 * int(radius) ** 2

print(area)
```

Given the *meal price* (base cost of a meal), *tip percent* (the percentage of the *meal price* being added as tip), and *tax percent* (the percentage of the *meal price* being added as tax) for a meal, find and print the meal's *total cost*.

Input			Output
12.00	20	8	The total meal cost is 15 dollars.

Given:

$$mealCost = 12, tipPercent = 20, taxPercent = 8$$

Calculations:

$$tip = 12 \times \frac{20}{100} = 2.4$$

$$tax = 12 \times \frac{8}{100} = 0.96$$

$$totalCost = mealCost + tip + tax = 12 + 2.4 + 0.96 = 15.36$$

$$round(totalCost) = 15$$

We round $totalCost$ to the nearest dollar (integer) and then print our result:

```
meal_cost = float(input("Enter meal Cost "))

tip_percent = int(input("Enter tip percent"))

tax_percent = int(input("Enter tax percent"))

# Write your calculation code here
tip = tip_percent / 100 * meal_cost
tax = tax_percent / 100 * meal_cost

# cast the result of the rounding operation to an int and save it as tota
total_cost = int(round(meal_cost + tip + tax))

# Print your result
print("The total meal cost is " + str(total_cost) + " rupees.")
```

- Temperature Conversion.
- Given the temperature in Celsius, convert it to Fahrenheit.
- Given the temperature in Fahrenheit, convert it to Celsius.

Relational operators

- Relational operators are used to compare values.
- It returns **True** or **False** according to condition

Operator	Meaning	Example
>	Greater that - True if left operand is greater than the right	x > y
<	Less that - True if left operand is less than the right	x < y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y

```
1    x = 10
2    y = 12
3
4    print('x > y  is',x>y)
5
6    print('x < y  is',x<y)
7
8    print('x == y is',x==y)
9
10   print('x != y is',x!=y)
11
12   print('x >= y is',x>=y)
13
14   print('x <= y is',x<=y)
```

```
x > y  is False
x < y  is True
x == y is False
x != y is True
x >= y is False
x <= y is True
```

```
>>> a = 20
>>> b = 10
>>> c = 8
>>> a > b > c
True
```

```
>>> age = 25
>>> 18 <= age <= 30
True
>>> age = 35
>>> 18 <= age <= 30
False
>>> age = 12
>>> 18 <= age <= 30
False
```

Logical operators

Operator	Meaning	Example
and	True if both the operands are true	x and y
or	True if either of the operands is true	x or y
not	True if operand is false (complements the operand)	not x

```
x = True
y = False

print('x and y is',x and y)

print('x or y is',x or y)

print('not x is',not x)
```

```
x and y is False
x or y is True
not x is False
```

Bitwise operators

- Bitwise operators operates on individual bits of number.

Operator	Meaning	Example
&	Bitwise AND	10 & 4 = 0
|	Bitwise OR	10 | 4 = 14
~	Bitwise NOT	~10 = -11
^	Bitwise XOR	10 ^ 4 = 14
>>	Bitwise right shift	10>> 2 = 2
<<	Bitwise left shift	10<< 2 = 42

Compound Assignment operators

Operator	Example	Equivatent to
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5
&=	x &= 5	x = x & 5
|=	x |= 5	x = x | 5
^=	x ^= 5	x = x ^ 5
>>=	x >>= 5	x = x >> 5
<<=	x <<= 5	x = x << 5

Precedence of Python Operators

Operators	Meaning
()	Parentheses
**	Exponent
+x, -x, ~x	Unary plus, Unary minus, Bitwise NOT
*, /, //, %	Multiplication, Division, Floor division, Modulus
+, -	Addition, Subtraction
<<, >>	Bitwise shift operators
&	Bitwise AND
^	Bitwise XOR
|	Bitwise OR
==, !=, >, >=, <, <=, is, is not, in, not in	Comparisions, Identity, Membership operators
not	Logical NOT
and	Logical AND
or	Logical OR

Thank You